

Guião de Apoio 7 ZooKeeper em Python (zkServer, zkCli)

1. Introdução

Neste guião, serão implementadas aplicações distribuídas que utilizam os serviços do ZooKeeper para coordenar os participantes da mesma.

O **ZooKeeper** é um servidor da Apache que é utilizado em aplicações distribuídas para **coordenar, sincronizar e gerir configurações** entre vários programas. Para compreender melhor o objetivo e utilidade do ZooKeeper, suponha que tem vários sistemas (por exemplo, vários servidores) a serem executados em simultâneo. Eles precisam de:

- **eleição do líder:** saber quem é o líder (decidir que servidor é que “manda”)
- **configuração centralizada:** partilhar informação relevante através do próprio Zookeeper
- **naming/descoberta de servidores:** saber onde se encontram os vários servidores e quais as suas responsabilidades e tolerar faltas e shutdowns dos servidores
- **sincronização:** garantir que a lógica de negócio está sincronizada entre servidores atualizando o estado da lógica de negócio mantida por cada servidor de uma forma eficiente

Neste contexto, o ZooKeeper tem o papel de “arbitro central” que ajuda a atingir todos os objetivos acima citados de forma eficiente e tolerante a falhas.

O ZooKeeper organiza os dados dos vários servidores monitorizados numa estrutura tipo árvore (semelhante a um sistema de ficheiros) (ver figura abaixo):

- Cada nó é designado por **znode** e deve ter um nome atribuído pelo gestor da árvore
- Cada znode pode armazenar pequenos dados (ip, porto, estado atual etc.).
- As várias entidades do sistema distribuído (clientes ou servidores) podem **subscriver o Zookeeper para serem notificadas de mudanças** (watchers). No projecto, tanto os servidores como os seus clientes devem fazer watch do servidor ZooKeeper.

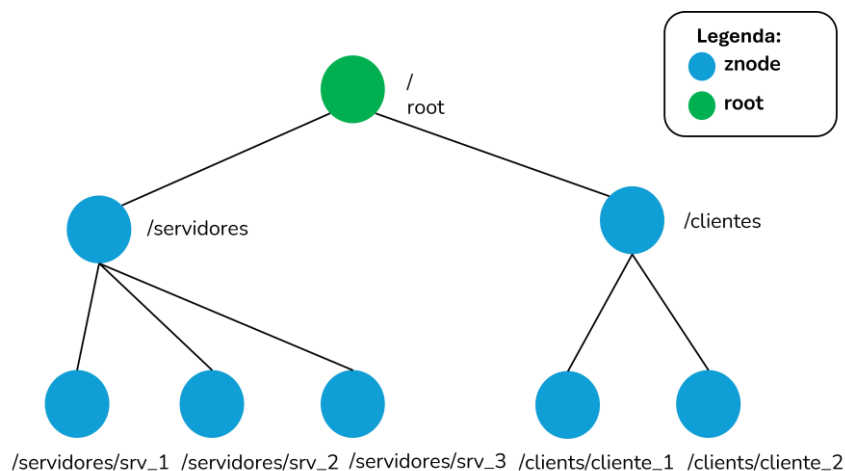


Figura 1: Estrutura hierárquica gerida pelo Zookeeper.

2. Exercícios

1º Exercício

Neste **exercício**, vamos aprender a verificar e gerir o estado do zookeeper.

1. Para executar o ZooKeeper **na sua máquina de laboratório (DI)**, deve seguir as instruções:

a) Mudar a diretoria corrente para a diretoria abaixo transcrita (usando o terminal)

```
cd /opt/zookeeper/bin/
```

b) Verificar o estado (status) do ZooKeeper (usando o terminal)

```
./zkServer.sh status
```

Comportamento Esperado:

Deve aparecer uma mensagem de similar à seguinte mensagem (deve ver **Mode: standalone**):

```
/usr/bin/java
ZooKeeper JMX enabled by default
Using config: /home/user/zookeeper/bin/./conf/zoo.cfg
Client port found: 2181. Client address: localhost. Client SSL: false.
Mode: standalone
```

Se receber a seguinte mensagem, o seu servidor ZooKeeper não está a funcionar:

```
/usr/bin/java
ZooKeeper JMX enabled by default
Using config: /home/user/zookeeper/bin/./conf/zoo.cfg
Client port found: 2181. Client address: localhost. Client SSL: false.
Error contacting service. It is probably not running.
```

2º Exercício

Neste **exercício**, vamos ligar-nos ao servidor Zookeeper do exercício 1 e observá-lo usando o **zkCli**. O **zkCli** (ZooKeeper CLI) é uma ferramenta de linha de comandos usada para interagir diretamente com o **Apache ZooKeeper**. Use **quit** para sair da aplicação.

2. Para executar o zkCli **na sua máquina de laboratório (DI)**, deve seguir as seguintes instruções:

- a) Mudar a diretoria corrente para a diretoria abaixo transcrita (usando o terminal)

```
cd /opt/zookeeper/bin/
```

- b) Corra o executável zkCli.sh.

```
./zkCli.sh
```

- c) Analisar a raíz da árvore de znodes.

```
ls /
```

- d) Descubra e aponte no caderno o significado de cada uma das flags do comando ls.

```
ls -R /
```

- e) Cria uma diretoria na raíz, de nome servidores_2526. Verifique o estado com comando ls -R /.

```
create /servidores_2526
```

- f) Crie um znode de nome servidor_1. Verifique o estado com comando ls -R /.

```
create /servidores_2526/servidor_1
```

- g) Modifique os dados do nó servidor_1.

```
set /servidores_2526/servidor_1 "127.0.0.1:4444"
```

- h) Obtenha os dados do nó servidor_1.

```
get /servidores_2526/servidor_1
```

- i) Apague o nó servidor_1. Verifique o estado após a eliminação.

```
delete /servidores_2526/servidor_1
```

3º Exercício

3. Neste **exercício** vamos ligar-nos ao Zookeeper através de um **cliente Python codificado por si** semelhante ao **ziCli**. Para este objetivo, necessita de inicializar um cliente Zookeeper que crie a árvore de **znodes** (/servers_2526) e o seu znode (/servers_2526/server_1).

- a) Numa **directoria à sua escolha**, coloque o ficheiro **clienteZoo3.py** dos ficheiros de suporte à aula. O ficheiro encontra-se abaixo transcrito. **Comente e imprima mensagens de interesse.**

```
# Cliente Zookeeper #

from kazoo.client import KazooClient

zk = KazooClient(hosts='127.0.0.1:2181') #porto default do Zookeeper
zk.start()

if not zk.exists("/servers_2526"):
    zk.create("/servers_2526")

if not zk.exists("/servers_2526/server_1"):
    zk.create("/servers_2526/server_1")

try:
    while True:
        pass
except KeyboardInterrupt:
    zk.stop()
```

- b) De seguida execute o código dentro da diretoria.

```
python3 clienteZoo3.py
```

- c) Use o **zkCli** para verificar a árvore de znodes.

```
./opt/zookeeper/bin/zkCli.sh
ls -R /
```

4º Exercício

4. Neste **exercício** vamos estruturar e estender um **novo cliente Python do ZooKeeper** para ter um comportamento semelhante ao **zkCli**. Para este objetivo, siga as instruções abaixo.

a) Analise o **clienteZoo4.py** dos ficheiros de suporte. O código encontra-se transcrito abaixo.

```
from kazoo.client import KazooClient
from kazoo.exceptions import NodeExistsError

# ligar ao ZooKeeper
zk = KazooClient(hosts="127.0.0.1:2181")
zk.start()
print("Ligado ao ZooKeeper")
# criar nó efémero
path = "/teste_ephemeral"
value = "127.0.0.1:4000"

try:
    created = zk.create(
        path,
        value.encode("utf-8"),
        ephemeral=True,
        makepath=True
    )
    print("Nó efémero criado:", created)
except NodeExistsError:
    print("O nó já existe.")

# listar raiz
children = zk.get_children("/")
print("Conteúdo de / :", children)

# fechar ligação
zk.stop()
zk.close()

print("Ligação fechada")
```

- b) De seguida execute o código dentro da diretoria.
c) Altere o código para implementar de forma interactiva o **ls create_ephemeral**.
d) Implemente os seguintes comandos para tornar a sua aplicação num zkCli simples:

```
python3 clienteZoo4.py
```

- ls /
- get /app
- create /app "ola"
- set /app "novo valor"
- delete /app
- exists /app
- stat /app
- cd /app

5º Exercício

5. Neste **exercício** vamos estruturar outro **cliente Python do ZooKeeper** que seja também um **servidor de outro cliente** para ter um comportamento semelhante ao **servidor e ao cliente do seu projeto**. Para este objetivo, siga as instruções abaixo.

a) Analise o **servidor5.py** dos ficheiros de suporte. O código encontra-se transcrito abaixo.

```
import json
import socket
import sys
from kazoo.client import KazooClient

zk = KazooClient(hosts="127.0.0.1:2181")
zk.start()

if len(sys.argv) == 3:
    ip = sys.argv[1]
    port = int(sys.argv[2])
else:
    print("Uso: python3 <executavel.py> <ip> <porto>")

if not zk.exists("/servers"):
    zk.create("/servers", b"")

server_id = zk.create(
    f"/servers/server_",
    json.dumps({"host": ip, "port": port}).encode(),
    ephemeral=True,
    sequence=True
)

s = socket.socket()
s.bind((ip, port))
s.listen(5)

while True:
    conn, addr = s.accept()
    msg = conn.recv(1024).decode()
    print(server_id, "recebeu:", msg)
    conn.send(f"{server_id} respondeu".encode())
    conn.close()
```

- b) Analise o `cliente5.py` dos ficheiros de suporte. O código encontra-se transcrito abaixo.

```
import json
import socket
import sys
from kazoo.client import KazooClient

zk = KazooClient(hosts="127.0.0.1:2181")
zk.start()

children = zk.get_children("/servers")

# Ordena lexicograficamente (funciona por causa dos zeros à esquerda)
sorted_nodes = sorted(children)

# Maior = último da lista
max_node = sorted_nodes[-1]

# ir buscar os dados ao ZooKeeper
data, _ = zk.get(f"/servers/{max_node}")

# converter bytes → dict
info = json.loads(data.decode())

s = socket.socket()
s.connect((info["host"], info["port"]))
s.send(b"ola")
print(s.recv(1024).decode())
s.close()
```

- c) Altere o `servidor5.py` dos ficheiros de suporte para o servidor criar a hierarquia quando for o primeiro servidor a iniciar e a árvore ainda não tiver sido gerada.

3. Bibliografia e outro material de apoio

[1] <https://kazoo.readthedocs.io/en/latest/>

[2] <https://kazoo.readthedocs.io/en/latest/api/recipe/barrier.html#kazoo.recipe.barrier.DoubleBarrier>